

# Lab 3 – Architectural Justification: Micro-Lending & Peer Credit Risk Assessor

## Architectural Choice

We chose a **Microservices Architecture** for the Micro-Lending & Peer Credit Risk Assessor system. The architecture decomposes the system into six independently manageable components: **Order & Loan Manager Component, Credit Risk Assessment Component, Payment Service Component, Database Component (ACID & Encrypted), Repayment & Schedule Component, and Notification Service Component**. Each component represents a distinct responsibility and communicates with other components through well-defined interfaces and dependencies.

## Reasons for Choosing Microservices

### 1. Independent Scaling:

The **Credit Risk Assessment Component** performs borrower risk evaluation using debt-to-income ratio, repayment history, and employment stability. This processing may require more computational resources than other services such as the **Notification Service Component**. Microservices allow individual components to be scaled independently according to their workload.

### 2. Fault Isolation and Independent Deployment:

The system separates important functions such as loan management, credit assessment, payment processing, repayment management, and notifications. With microservices, failures can be isolated between components, while individual components can be updated or deployed independently without requiring the entire system to be redeployed.

## Security Advantage

Microservices allow security responsibilities to be separated between services. The **Database Component (ACID & Encrypted)** protects sensitive borrower financial data such as income, debt, and repayment history through **encryption at rest and in transit**, with role-based access control restricting access to authorized information. Other components access the database through defined interfaces, keeping internal implementation details protected.

## Performance Advantage

The **Database Component (ACID & Encrypted)** supports loan repayment transactions and EMI distribution using **ACID transactions**, helping prevent double-spending, duplicate postings, and lost transactions during concurrent operations. In addition, microservices allow resource-intensive operations such as **credit-risk assessment** to be scaled independently according to workload.